

Final Report

Model Extraction: Stealing a Black-Box Classifier (Prediction-API Model Stealing)

Group ID	9 (Section B2)
Group members	Pritu Dhar Dhip (2105109) Arian Mahtab Chowdhury (2105106)
Course	CSE406: Cyber Security Sessional
Submission Date	18th September, 2026

1 Recap: What Was Built and Who Owns What

This report is the companion to the *Design Report* (11th September, 2026) for the same project: a model-extraction attack against a secret MNIST classifier served over a custom TCP prediction API, plus the layered defences in Section 6 of that report. Nothing about the design changed; this document records what actually happened when the design was built and run end to end.

Component	Owner	File
Wire protocol (12-byte header, framing)	shared	<code>protocol.py</code>
Victim server, defence layer, Attack B (equation-solve)	Pritu Dhar Dhip	<code>victim/</code>
Query engine, Attack A (knockoff distillation)	Arian Mahtab Chowdhury	<code>attacker/</code>

All runs below use `SEED = 1337` (NumPy, PyTorch, Python `random`), the same victim checkpoint (`victim.pt`, trained once by `train_victim.py`), and the attacker/victim on `127.0.0.1` (same host, two processes – the design report’s threat model of “two hosts on one segment” realised as two local processes for the demo). Every command below is exactly the command in `README.md`; nothing was modified to make the numbers look better.

There is no graphical UI on either side – both the victim and the attacker are command-line processes – so “victim screen” / “attacker screen” below means the two processes’ console output, captured verbatim in `results/logs/experiments.log` and quoted throughout this report.

2 Attack A: Learning-Based Extraction (Knockoff Distillation)

2.1 Attack Process

1. **Victim side (Pritu).** Train the secret VictimCNN once on MNIST and save `victim.pt`:

```
cd victim
python train_victim.py --epochs 5 --out victim.pt --data-dir ../attacker/data
```

Start the server, undefended:

```
python victim_server.py --host 127.0.0.1 --port 9009 --model victim.pt
```

2. **Attacker side (Arian).** Run the black-box query engine for a budget $q \in \{1000, 5000, 10000, 30000\}$, using MNIST test images as the (non-adaptive, fixed-in-advance) query pool:

```
cd attacker
python attack_client.py --host 127.0.0.1 --port 9009 --pool mnist --queries 10000 --out b10000.npz
```

Each query is one REQUEST/RESPONSE round trip: the attacker never sees a weight or a gradient, only a probability vector.

3. **Offline distillation (Arian, no network).** Train a substitute SmallCNN on the recorded (x, p) pairs via temperature-scaled soft-label distillation:

```
python knockoff_train.py --transfer b10000.npz --epochs 20 --T 2.0 --out k_b10000.pt
```

4. **Repeat** step 2–3 with an out-of-distribution query pool (`--pool fashion`) and with the victim’s defences turned on (`--top-k 3 --noise-std 0.02, --label-only`) to measure their effect on the same budget sweep. The rate limiter is evaluated separately, at a much smaller quota (`--rate-limit 50 --rate-window 60`) chosen purely to make the demonstration fast and easy to reproduce – see Section 4.3; the mechanism being demonstrated (a hard per-client cutoff at `ERROR code 4`) does not depend on the specific quota value.

2.2 Expected Outcome (from the Design Report, Section 5)

Condition	Expected result
In-distribution (MNIST) queries, 10k–30k budget	$\hat{f} \approx 0.90\text{--}0.98\times$ victim accuracy
Out-of-distribution (Fashion-MNIST) queries	still usable, $\gtrsim 0.8\times$
Fidelity (label agreement with victim)	high but $< 100\%$, bounded $\approx 93\text{--}94\%$
Adversarial examples crafted on $\hat{f}$	fool the victim well above chance

2.3 A Measurement Pitfall We Had to Correct First

`attack_client.py`’s `--pool mnist` draws its query images from the MNIST *test* split (`train=False`), which has exactly 10,000 images. `knockoff_train.py`’s own built-in accuracy check evaluates against that same split. At $q = 10000$ or $q = 30000$ this means the attacker’s query pool *is* (almost) the entire evaluation set: fidelity computed that way is agreement on images the knockoff already saw labelled by the victim during training, not a fair generalisation test. We evaluate all numbers below on a disjoint 3,000-image sample of the MNIST *train* split instead (which the attacker never queries), giving an unbiased accuracy/fidelity number. This single measurement choice turned out to matter more than any defence setting we tested – see Section 2.4.

A second consequence of the same 10,000-image ceiling: at $q = 30000$, `build_query_pool` can only draw 10,000 real images from the test split, so it silently pads the remaining 20,000 queries with synthetic uniform-noise images (`attack_client.py`’s own documented fallback, “top up with noise if the test set is smaller”). We confirmed this directly: pixel statistics of `b30000.npz` show indices 0–9,999 matching real MNIST digits (86–90% of pixels within 0.05 of 0 or 1, the bimodal ink-on-background signature) and indices 10,000–29,999 matching uniform noise (mean ≈ 0.5 , std $\approx 0.289 = 1/\sqrt{12}$) almost exactly. So the “30,000-query” condition in Section 2.4 is really 10,000 real in-distribution queries plus 20,000 noise queries, not 30,000 in-distribution queries – which is the more direct explanation for why its accuracy/fidelity barely exceeds the $q = 10000$ result (Section 2.6): there is no additional real digit supervision past 10,000 queries for this pool, only more views of the victim’s behaviour on inputs off the data manifold.

2.4 Actual Outcome

The victim’s own test accuracy on the full 10,000-image MNIST test set is **99.14%**. All fidelity/accuracy figures below are against the disjoint 3,000-image train-split holdout described above.

Budget q	Undefended		Top-3 + noise (0.02)	
	Accuracy	Fidelity	Accuracy	Fidelity
1,000	0.9143	0.9143	0.9157	0.9157
5,000	0.9747	0.9753	0.9767	0.9770
10,000	0.9850	0.9863	0.9840	0.9840
30,000	0.9843	0.9857	0.9830	0.9823

Condition (10k budget)	Accuracy	Fidelity
Label-only (extreme case, $k=1$)	0.9830	0.9827
Out-of-distribution pool (Fashion-MNIST)	0.8363	0.8383

FGSM adversarial-example transfer (crafted white-box on the knockoff, then replayed against the real victim, unmodified):

Knockoff	$\varepsilon = 0.05$		$\varepsilon = 0.1$		$\varepsilon = 0.2$	
	fool $\hat{f}$	transfer	fool $\hat{f}$	transfer	fool $\hat{f}$	transfer
Baseline, 10k	0.054	0.011	0.145	0.070	0.613	0.481
Defended, 10k	0.052	0.009	0.125	0.045	0.466	0.487
Label-only, 10k	0.059	0.009	0.161	0.037	0.497	0.340
OOD (fashion), 10k	0.418	0.012	0.678	0.126	0.911	0.513

$fool \hat{f}$ = white-box FGSM success rate against the knockoff itself; $transfer$ = fraction of those adversarial images that also change the real victim’s prediction relative to its own clean prediction. Chance level for a 10-class problem is 10%; transfer at $\varepsilon = 0.2$ reaches 34–51%, well above chance, confirming the design report’s claim.

2.5 Observed Console Output

Attacker console (undefended, $q = 10000$, excerpt):

```
[client] query pool: mnist (10000 images)
[client] 1000/10000 queries (1171 q/s) last argmax=9
...
[client] 10000/10000 queries (1172 q/s) last argmax=6
[client] campaign done: 10000 pairs in 8.8s, flags observed = [1]
[client] defences advertised by responses: {'probs_present': True, 'topk_truncated': False, 'noised': False}
```

Victim console, defended ($k=3$, noise= 0.02), same campaign at $q = 500$:

```
[victim] listening on 127.0.0.1:9009 (model=trained, defence={'round_decimals': None,
'top_k': 3, 'label_only': False, 'noise_std': 0.02, 'rate_limit_queries': None,
'rate_limit_window_s': 60.0})
[victim] 127.0.0.1 disconnected after 500 queries (quota used: 0)
```

The attacker’s own console for that same run reports `flags observed = [7]`, decoded by `attack_client.py` as:

```
{'probs_present': True, 'topk_truncated': True, 'noised': True}
```

confirming, straight off the wire, that the defence was active and the client could see so, exactly as Section 3.4 of the design report intended.

Offline distillation (attacker console, excerpt from the $q = 10000$ run):

```
[train] epoch 01/20 loss=4.0932 val_fidelity=0.8970
[train] epoch 02/20 loss=1.3969 val_fidelity=0.9520
...
[train] epoch 20/20 loss=0.5010 val_fidelity=0.9890
[train] knockoff test accuracy vs ground truth = 0.9916
```

2.6 Was the Attack Successful? Why / Why Not?

Yes, against every configuration it actually completed a budget against. Excluding the rate limiter (addressed below, where it is cut off by design before finishing), Attack A reconstructs a substitute that agrees with the victim 91–99% of the time across every in-distribution budget (1k–30k) and every defence configuration we tested (undefended, top-3 + noise, label-only); the out-of-distribution pool (Fashion-MNIST) is noticeably weaker at 84%, but still clearly usable, matching the design report’s own “ $\gtrsim 0.8\times$ ” expectation rather than contradicting it. Its adversarial examples transfer to the real victim well above chance in every case. It succeeds because the victim’s response is, by construction, a full supervised training signal: every query hands the attacker exactly the label (and, undefended, the confidence) it would need to train a classifier directly, and `victim_server.py` answers every well-formed query regardless of who is asking.

Against the rate limiter (Section 4.3), Attack A did *not* succeed at reaching its requested budget – not because the extracted model was worse, but because the campaign was cut off by design before it could finish. Our first run also exposed a genuine implementation bug on the attacker side: the original `attack_client.py` had no handling for an `ERROR` frame and crashed on it, discarding every query already collected, not just the ones after the quota. We fixed this (`attack_client.py` now catches the rate-limit error, stops, and saves whatever it already has) and reran the demonstration – see Section 4.3 for the corrected behaviour. That gap was an attacker-side robustness bug, not evidence the model itself resists extraction; a patient attacker that simply waits out repeated windows is not stopped by this mechanism, only slowed.

Three results differ from the Design Report’s Section 5 expectations, and all three are informative rather than a sign of a broken attack:

- At $q \geq 5000$, in-distribution accuracy/fidelity (0.97–0.99) sit at or slightly *above* the predicted $0.90\text{--}0.98\times$ band. MNIST is an easy, low-ambiguity 10-class task for a 99%-accurate teacher, so temperature-scaled distillation over 20 epochs converges further than the conservative estimate assumed.
- Going from $q = 10000$ to $q = 30000$ barely moves accuracy/fidelity at all (Section 2.4). The more direct cause is Section 2.3’s second finding: 20,000 of those 30,000 queries are synthetic noise, not additional real digit supervision, so there is little new signal to learn from past the first 10,000 queries for this pool – this, not only distillation saturating, is why the curve flattens.
- Section 2.4 is the bigger surprise: the tested defence settings barely move these numbers at all.

3 Attack B: Equation-Solving Extraction (Linear/Softmax)

3.1 Attack Process

`equation_solve.py` (Pritu) recovers a linear/softmax victim’s weights directly, with no training loop: for a genuine softmax model, $\log p_i(x) - \log p_0(x) = (w_i - w_0) \cdot x + (b_i - b_0)$ is *exactly* linear in x , so one batch of $(d + 1)$ non-adaptive probe queries plus one ordinary-least-squares solve recovers w_i, b_i up to the softmax’s gauge freedom ($w_0 = 0, b_0 = 0$).

```
# Sanity check: a genuine linear/softmax victim (no network)
python equation_solve.py --mode offline --target linear

# Diagnostic: the real trained CNN, offline (linearity assumption should break)
python equation_solve.py --mode offline --target cnn --checkpoint victim.pt --data-dir ../attacker/
data

# Online: the real victim_server.py, undefended, then defended
python equation_solve.py --mode network --host 127.0.0.1 --port 9009 --data-dir ../attacker/data
```

3.2 Expected Outcome

Target	Expected result
Genuine linear/softmax victim	$\approx 100\%$ label agreement, a few hundred queries
Real trained CNN (<code>victim.pt</code>)	fidelity collapses toward chance ($\approx 10\%$ on MNIST)

This is deliberate: Attack B’s whole method assumes the victim’s logits are *globally* linear in the input, which holds exactly for a softmax/logistic model and not at all for a ReLU CNN.

3.3 Actual Outcome

Every run below queries $d + 1 = 785$ probes to solve for the 7,065 unknowns (9×785), then evaluates fidelity on 200 real MNIST test images, exactly as `equation_solve.py` is written to do.

Mode	Target	Fidelity	Mean TV distance
Offline	Genuine linear/softmax victim	1.0000	0.000001
Offline	Real CNN (<code>victim.pt</code>)	0.0500	0.949965
Network	Real server, undefended	0.0500	0.949965
Network	Real server, top-3 + noise (0.02)	0.1000	0.891578

The network-mode run against the undefended server reproduces the offline CNN diagnostic’s numbers exactly (0.0500 fidelity both times) – an independent confirmation that `protocol.py`, `victim_server.py`, and `equation_solve.py` are correctly wired end to end, per the victim README’s own verification claim.

3.4 Was the Attack Successful? Why / Why Not?

Only against the target it was designed for. Against a genuine linear/softmax victim, Attack B is a complete success: near-perfect fidelity and vanishing total-variation distance from under 800 non-adaptive queries and no training loop at all. Against the real `VictimCNN`, it fails exactly as the design report predicted: fidelity collapses to chance level (10 classes $\Rightarrow \approx 10\%$; we measure 5–10%). The reason is architectural, not a bug: the least-squares solve assumes $\log p_i(x) - \log p_0(x)$ is *globally* linear in x , which is exact for a softmax/logistic model and false for a ReLU CNN – the

recovered weights fit whichever single activation region the probe batch happened to land in, and that fit does not generalise to real digit images.

Turning the top-3 + noise defence on moves the CNN fidelity from 0.05 to 0.10 – both are noise around the 10% chance floor, so this run cannot show the defence doing anything measurable to Attack B: the attack is already destroyed by the victim’s non-linearity before the defence has anything left to degrade. This matches the design report’s own framing of Attack B as a special-case diagnostic, not a general threat to this particular victim.

4 Defence Evaluation

Defences live in `victim/defence.py` and are evaluated on vs. off against the *same* query budgets as Section 2, so the accuracy/fidelity cost of each defence can be read directly off the two curves.

4.1 Output Minimisation (top- k , label-only)

Mechanism: `defence.py::apply_top_k` zeroes every class probability outside the top- k before the response is sent; `--label-only` is the $k=1$ extreme (a one-hot vector at the argmax). Both set the `FLAG_TOPK` wire bit so the attacker’s client can see, from the response alone, that this defence is active (Section 2.5’s console excerpt shows exactly this).

Expected (design report): “rounding to 2–3 decimals or dropping confidences sharply raises the query cost of learning- and equation-based attacks.”

Actual: at $k=3$, Attack A’s accuracy/fidelity moved by at most 2 percentage points at any budget (Section 2.4); at $k=1$ (label-only, the most extreme setting in the codebase) it moved by *less than one point* from the undefended baseline at the same 10k budget (0.983 vs. 0.986 fidelity). This does *not* match the qualitative expectation.

4.2 Prediction Perturbation (calibrated noise)

Mechanism: `defence.py::apply_perturbation` adds $\mathcal{N}(0, \sigma^2)$ noise to the probability vector, renormalises, and retries at a damped σ (up to 3 times) if the noise would flip the argmax – an honest user’s label never changes, but the soft-label shape an attacker trains on is perturbed.

Expected: “corrupt[s] the soft-label signal the distillation attack relies on, lowering the knock-off’s fidelity.”

Actual: combined with top-3 truncation at $\sigma = 0.02$, no budget showed a fidelity drop larger than 2 points versus undefended (Section 2.4).

4.3 Why Output Minimisation and Noise Underperformed Here

Both defences work exactly as unit-tested in `defence.py`’s self-test (argmax preserved, valid distribution, correct flag bits) – the mechanism is not broken. What the design report’s qualitative Section 6 discussion did not anticipate is how *peaked* a 99%-accurate MNIST softmax output already is for almost every digit image: one class near 1, the rest near 0. Keeping only the top-3 classes discards almost no probability mass on such an input, and $\sigma = 0.02$ noise is tiny next to the gap between the top class and the rest – `apply_perturbation`’s own argmax-preserving retry logic actively damps it further. Even label-only survives because the query pool is natural, unambiguous MNIST digits: a one-hot label is nearly as informative as the full distribution when the underlying task has almost no class confusion for the victim to begin with. The “dark knowledge” these defences are designed to strip out matters most on *ambiguous* inputs, which a 99%-accurate model rarely produces here. At the parameter values tested, output minimisation and perturbation

are real but weak frictions on this particular victim, not the “sharp” cost increase the qualitative discussion suggested – a stronger setting ($\sigma \gg 0.02$, or k combined with heavier rounding) would likely be needed to see the predicted effect, at the cost of also degrading the response for honest users.

4.4 Query-Pattern Monitoring / Rate Limiting

Mechanism: `defence.RateLimiter` is a per-client (peer IP) sliding window; once a client’s window is full, `victim_server.py` returns an `ERROR` frame with code 4 instead of a prediction, for every further request in that window.

Expected (victim README): “confirmed the client receives `ERROR code 4` once the quota is exceeded.”

Actual: exactly that – with `--rate-limit 50 --rate-window 60`, the server served precisely 50 predictions and then returned code 4 on the 51st, out of a requested 200:

```
[victim] listening on 127.0.0.1:9012 (... 'rate_limit_queries': 50, 'rate_limit_window_s': 60.0})
[victim] 127.0.0.1 disconnected after 50 queries (quota used: 50)
```

The defence itself worked as designed. What was *not* expected: our *first* run used the original `attack_client.py`, which has no exception handling around this error – the whole script crashed (`RuntimeError: victim returned ERROR code 4, exit code 1`) and `main()` never reached the `np.savez_compressed` call, so every query collected before the block, not just the ones after it, was lost. That was an attacker-tooling bug, not a defence weakness, so we fixed it: `attack_client.py` now catches the rate-limit error specifically, stops the campaign, and keeps whatever it already collected. Re-running the identical command with the patched client:

```
[client] quota exceeded after 50 queries (victim returned ERROR code 4) --
        stopping, keeping what was already collected
[client] campaign done: 50 pairs in 0.1s, flags observed = [1] (stopped early: rate limited)
[client] wrote transfer set -> ratelimit_fixed.npz
```

exit code 0, 50 usable pairs saved. A defender should still assume a competent attacker would do exactly this – catch the error, keep what it already has, and simply wait out the window or rotate identity, which the rate limiter’s own documented caveat (“a non-adaptive attack using natural inputs can evade distribution-based detection”) already concedes it cannot stop by itself. Fixing our own client only made this limitation more visible, not less real.

4.5 Did Each Defence Work as Expected?

Partially. Rate limiting behaved exactly as designed and is the one control that produced a hard, unambiguous stop. Output minimisation and prediction perturbation both function correctly at the mechanism level (verified by `defence.py`’s own self-test and by the `FLAGS` bits on the wire) but, at the specific settings tested against this specific victim, degraded Attack A far less than the design report’s qualitative discussion implied – including at the label-only extreme. The honest conclusion is closer to, but stronger than, the design report’s own: *no single defence stops extraction, and on an easy, low-ambiguity task like MNIST, output minimisation and light perturbation raise the attacker’s cost by only a few percentage points of fidelity, not enough to change the attack’s practical success*. Rate limiting is what actually imposes a real cost, and even that only stops a single, non-resuming client.

5 Assumptions

- The victim and attacker are two independent processes on the same network segment (localhost, in this demo); the attack does not depend on eavesdropping a third party’s traffic – it is an authorised client of its own TCP socket.
- The victim, by default, returns a full class-probability vector per query (the common MLaaS pattern) rather than a label only; the label-only case is evaluated separately in Section 4 as the harder variant.
- The attacker’s query set is *non-adaptive* (fixed in advance from a public pool – MNIST test images, Fashion-MNIST images, or synthetic uniform noise) and is not crafted to look like organic traffic beyond using natural images; a determined defender inspecting raw query volume could in principle notice a query burst even though the images themselves look ordinary.
- Rate limiting is keyed by peer IP; an attacker that rotates source addresses is out of scope for the rate-limiter’s protection, per the design report’s own caveat (Section 6.3).
- The wire itself carries no transport encryption (no TLS) – irrelevant to model extraction, which only needs the plaintext prediction values the server willingly returns, not a man-in-the-middle view of someone else’s traffic.
- The attacker does not know the victim’s architecture; the substitute `SmallCNN` architecture is chosen independently, consistent with the design report’s premise that only *functionality*, not weights, is being copied.
- All results are seeded (`SEED = 1337`) for reproducibility; the reported numbers are single runs, not averages over multiple seeds – small run-to-run variance (data-loader shuffling, distillation non-determinism) is expected and is discussed where it affects a conclusion.
- The in-distribution query pool (`--pool mnist`) and the MNIST test split are the same 10,000 images, so any fidelity/accuracy number evaluated against that split is contaminated once the query budget approaches it (Section 2.3). We assume a disjoint holdout (here, a sample of the MNIST train split) is the correct way to measure generalisation, and report all such numbers that way rather than using `knockoff.train.py`’s own built-in metric directly.

6 Conclusion

Attack A and Attack B behaved consistently with their core design assumptions, although several measured outcomes differed from the quantitative expectations in the Design Report. Attack A (learning-based) succeeds at its actual goal, the real `VictimCNN`: a 91–99%-fidelity substitute from nothing but the prediction API, across every in-distribution budget it was allowed to complete (rate limiting cuts it off by design, not by degrading it – Section 2.6), and its adversarial examples transfer to the real victim well above chance. Attack B (equation-solving) succeeds only against a genuine linear/softmax victim – near-perfect, in under 800 queries, no training loop – and, against that same real CNN, fails exactly as predicted, collapsing to chance level. Both outcomes were measured, not assumed, and both confirm the design report’s expected-outcome tables rather than contradicting them, even where the measured numbers differ in degree (Section 2.6). Attack B’s CNN failure is a confirmation of its stated assumptions, not a bug – the same distinction the design report itself draws between the two attacks. The defence layer stops the equation-solving attack

for free (it never worked against a CNN to begin with) but only meaningfully stops the learning-based attack through rate limiting; output minimisation and calibrated noise, at the settings we tested, are real but weak friction on an easy, low-ambiguity task like MNIST digit classification. The realistic goal the design report set out to demonstrate – making stealing more expensive than paying – is only partly met here: a determined attacker that simply waits out the rate limit’s window across multiple sessions – exactly what our own (now-fixed) client is capable of doing, one window at a time – would still walk away with a near-complete copy of the model.

References

1. F. Tramèr, F. Zhang, A. Juels, M. K. Reiter, T. Ristenpart. *Stealing Machine Learning Models via Prediction APIs*. USENIX Security 2016.
2. M. Jagielski, N. Carlini, D. Berthelot, A. Kurakin, N. Papernot. *High Accuracy and High Fidelity Extraction of Neural Networks*. USENIX Security 2020.
3. T. Orekondy, B. Schiele, M. Fritz. *Knockoff Nets: Stealing Functionality of Black-Box Models*. CVPR 2019.

See the companion Design Report (11th September, 2026) for the full threat model, protocol specification, and design justification this report builds on.